

Annexes

Panique en Parapente

List of Annexes

A	Mathematical models	3
B	Gantt	5
C	Cahier des charges	6
C.1	Primaires	6
C.2	Secondaires	6
C.3	Nice-to-have	6
C.4	Tâches	6
D	Journal de travail	8
E	User guide	20
F	Code documentation	21

A Mathematical models

Haversine formula

The haversine distance d helps us to account for the curvature of the Earth by getting the distance between two points given their latitudes and longitudes. We cannot use the Euclidean distance L_2 as it would produce errors for large distances. We will pose x, y as latitude and longitude (in radians) respectively. Their differences are calculated as such:

$$\Delta x = x_{\text{wp}} - x_{\text{user}}$$

$$\Delta y = y_{\text{wp}} - y_{\text{user}}$$

The angle a is computed from the Haversine function:

$$a = \sin^2\left(\frac{\Delta y}{2}\right) + \cos(y_1) \cdot \cos(y_2) \cdot \sin^2\left(\frac{\Delta x}{2}\right)$$

The angular distance c (also in radians) is as follows:

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

The distance d in meters is gotten by multiplying Earth's radius R with the angular distance:

$$d = R \cdot c$$

with $R = 6,371 \cdot 10^6$ meters, the mean radius of the Earth, as defined for the ellipsoid for the WGS84 coordinate system.

Bresenham line algorithm

We use both the generalized version and the integer one, which permits us to go in all octants, with better performance and no floating point errors.

The algorithm proceeds in four steps:

1. Calculation of absolute deltas,

We first determine the absolute distance the line must travel along each axis:

$$\Delta x = |x_{\text{End}} - x_{\text{Start}}|$$

$$\Delta y = |y_{\text{End}} - y_{\text{Start}}|$$

2. Determination of direction,

The step direction (s_x, s_y) is determined based on the start and end coordinates:

- If $x_{\text{Start}} < x_{\text{End}}$, then $s_x = 1$, else $s_x = -1$.
 - If $y_{\text{Start}} < y_{\text{End}}$, then $s_y = 1$, else $s_y = -1$.
3. Error initialization,
The error term ε is initialized to balance the movement between the dominant and non-dominant axes:

$$\varepsilon = \Delta x - \Delta y$$

4. Incremental loop,
The algorithm iterates from the start point to the end point. At each step, a temporary error variable e_2 is calculated to determine if a step should be taken along x , y , or both:

- Let $e_2 = 2 \cdot \varepsilon$
- Step in X: If $e_2 > -\Delta y$:

$$\varepsilon \leftarrow \varepsilon - \Delta y$$

$$x \leftarrow x + s_x$$

- Step in Y: If $e_2 < \Delta x$:

$$\varepsilon \leftarrow \varepsilon + \Delta x$$

$$y \leftarrow y + s_y$$

This approach ensures that the algorithm selects the tiles that best approximate the linear path between the pilot and the waypoint, with $O(1)$ complexity per step.

B Gantt

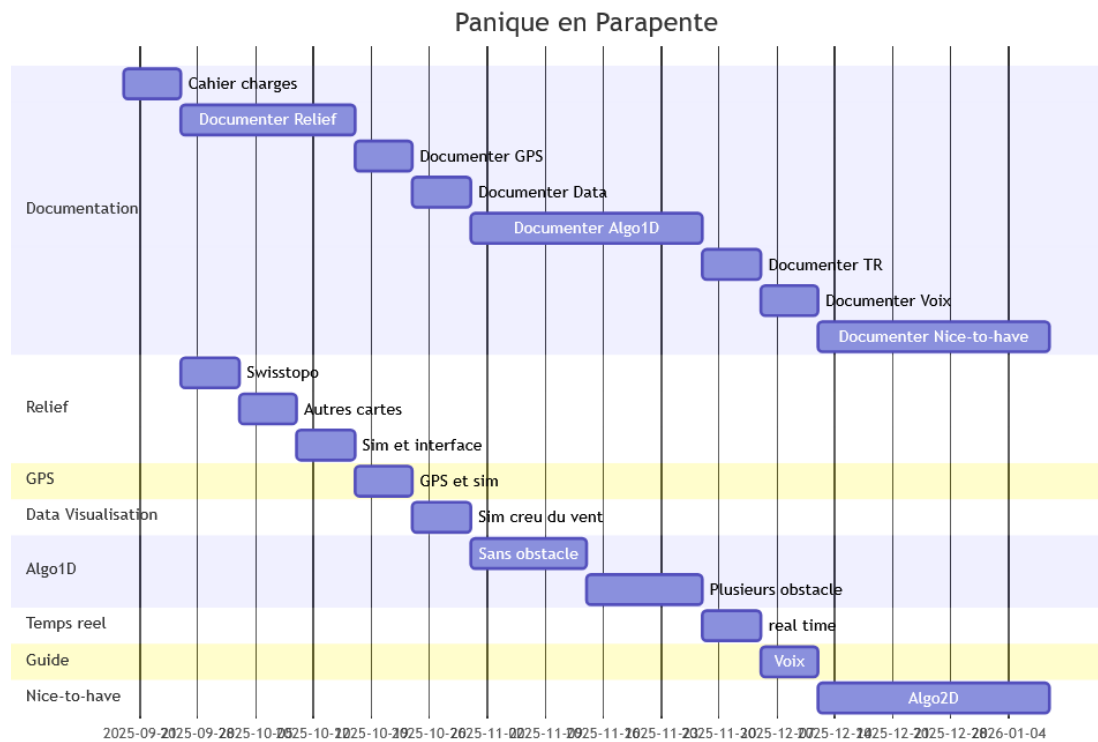


Figure 1: Gantt chart

C Cahier des charges

C.1 Primaires

1. Développement de l'algorithme Algo1D.
2. Interface pour pouvoir accueillir des topologies provenant de services différents
3. Application doit marcher OFFLINE

C.2 Secondaires

1. Voix qui guide le pilote (appel le delta qu'il faut monter, le delta par rapport au Waypoint et la direction du Waypoint)
2. Visualiser le tile du creux du vent (en couleur HSB) pour tester l'interface

C.3 Nice-to-have

1. Mode contournement : amélioration de l'algorithme qui permet le pilote de contourner les obstacles du relief.
2. Pouvoir faire marcher en dehors de la suisse

C.4 Tâches

1. **Relief** : Chercher les topologies existantes:
 - (a) Swisstopo
 - i. Vérifier license
 - ii. Vérifier si API (pour offline)
 - iii. Vérifier précision du maillage
 - iv. Vérifier système de coordonnées
 - v. Vérifier si format Little Endian / Big Endian
 - (b) Autres (ex: package Dart?)
 - (c) Créer une simulation de plusieurs reliefs (matrice 2D?):
 - i. Plat
 - ii. Un obstacle entre GPS et Waypoint

- iii. Plusieurs obstacles entre GPS et Waypoint
 - (d) Créer une interface **ServiceElevation_I** qui permet de recevoir n'importe quel topologie.
 - (e) ATTENTION à la luminosité des images de topologie
- 2. **GPS** :
 - (a) Sur dart, chopper la position du pilote du GPS de son natel
 - (b) Voir comment on peut faire marcher le GPS offline
 - (c) Créer des points GPS de simulation pour tester
- 3. **Data Visualisation** : Visualiser point gps ET tile de la carte de topologie sur une carte2D (coder en HSB) au creux du vent
- 4. **Algo1D** : Créer algorithme qui tire une ligne droite de la position du GPS en direction du waypoint en tenant compte de la finesse du parapente. Retourne le delta du GPS (de combien le pilote doit monter) et le delta du waypoint (de combien de mètre il va être en différence quand il atteint le Waypoint)
 - (a) Coder sans obstacles
 - (b) Coder pour un obstacle
 - (c) Coder pour plusieurs obstacles
- 5. **Temps réel** : Faire marcher (chaque x secondes) l'algorithme avec le GPS variable
- 6. **Guide** : Implémenter une voix (Text-to-Speech?) qui va indiquer les deux deltas et la direction à prendre pour aller au Waypoint
ex: "Climb 20 meters."
- 7. *Nice-to-have*, chercher des topologies qui marche en dehors de la Suisse
- 8. *Nice-to-have*, **Algo2D** : Modifier l'algorithme pour pouvoir contourner des obstacles (spline? polygoniser les courbes de bezier)

D Journal de travail

24/09/2025

Relief

J'ai trouvé une API pour swisstopo
swissSURFACE3D Raster Très dense, faut regarder si on peut fetcher des tiles en utilisant l'api (par exemple, récupérer tout les tiles d'un rayon x depuis le Waypoint)

Pour l'utiliser dans l'application, il faut l'attribuer correctement: <https://www.swisstopo.admin.ch/fr/conditions-utilisation-geodonnees-et-geoservices-gratuit>

Coordonnées: CH1903+ / MN95

25/09/2025

GPS

sur dart, package géolocator pour capter la position GPS du natel

Nice-to-have

Peut-être rajouter l'effet du vent? Difficile à trouver un API qui peut aider.
Voir Ce site, aviation weather API

26/09/2025

Dart

Pour les éléments UI, il faudrait peut-être faire du Dart + Flutter.

27/09/2025

Dart

Flutter sera utilisé pour tester les interfaces/factory de Dart, mais je ne pense pas qu'il devrait figurer dans l'implémentation futur de l'application, vu que ce n'est utilisé que pour tester

30/09/2025

Algo1D

Coordonnées Waypoint (static, input par utilisateur) :

$$f(x, y, z)_{\text{wp}} = \begin{pmatrix} x_{\text{wp}} \\ y_{\text{wp}} \\ z_{\text{wp}} \end{pmatrix}$$

Coordonnées Utilisateur (dynamique, input par GPS chaque t seconde) :

$$g(x, y, z)_{\text{user}} = \begin{pmatrix} x_{\text{user}} \\ y_{\text{user}} \\ z_{\text{user}} \end{pmatrix}$$

Fonction affine depuis la finesse du parapente (ici, $F = 9$) :

$$\Delta_{1D}(d, z_{\text{user}}, z_{\text{wp}}) = -\frac{1}{F}d + z_{\text{user}} - z_{\text{wp}} - \epsilon$$

avec distance d euclidienne, entre le Waypoint et le parapentiste:

$$d(x_{\text{wp}}, x_{\text{user}}, y_{\text{wp}}, y_{\text{user}}) = \sqrt{(x_{\text{wp}} - x_{\text{user}})^2 + (y_{\text{wp}} - y_{\text{user}})^2}$$

et la marge d'erreur ϵ pour s'assurer que le parapentiste soit en dessus du Waypoint (marge négative).

Cette fonction Δ_{1D} permet de récupérer le delta du parapentiste avec le Waypoint chaque t seconde.

Si cette fonction est négative, alors le parapentiste doit monter au moins le delta qui manque avec le Waypoint.

(1D car elle ne retourne seulement la hauteur)

Relief

Pour diminuer les téléchargements, nous pourrions garder les anciens tiles si on part du principe qu'un parapentiste a envie de refaire les mêmes trajets.

Pour la récupération de l'API STAC, il faut créer un bounding box avec un système de coordonnées WGS 84 à partir du Waypoint et le rayon r choisit par l'utilisateur.

Ex: <https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3d/items?bbox=7.43,46.95,7.69,47.10>

Dart

Pour développer sur android, il faut installer l'android SDK

01/10/2025

Algo1D

Bonus de sécurité:

créer un carré qui représente la largeur et longueur du parapentiste et faire l'algo1D pour chaque point du carré. Dans le cas d'un pont par exemple, on ne veut pas que le parapentiste y passe sous si sa forme entière ne peut pas passer

Relief

Pour le bounding box, il faut prendre en compte la courbure de la terre, sinon on a pas un carré comme il faut vu que la projection de la Terre sur les cartes amplifie les distances si on se rapproche des pôles (et la distance diminue réellement). Donc la forme d'un bounding box en Norvège n'aura pas la même allure qu'une au Kenya par exemple.

Si on a les coordonnées WGS84 de l'ouest, nord, est et sud que nous obtenons du rayon relativement au waypoint, nous devons calculer les deltas de latitude et longitude:

$$\Delta_{lat} = \frac{r_{km}}{111.32}$$
$$\Delta_{lon} = \frac{r_{km}}{111.32 \times \cos(\phi)}$$

avec ϕ = la latitude en radians:

$$\phi = \text{latitude} \times \frac{\pi}{180}$$

et 111.32 qui est la distance moyenne par degré de latitude:

$$\frac{40'0075}{360} = 111.32\text{km}$$

À l'équateur, les lignes longitudinales sont équidistantes avec les lignes latitudinales, où un degré correspond à ~ 110.57 km. Aux pôles, un degré correspond à ~ 111.70 km, d'où la moyenne de 111.32 km.

Un exemple pour la différence engendrée par ceci:

À l'équateur (=0 degré): $\cos(0)=1$, donc 111.32 km/degré

À 60 degré de latitude: $\cos(60 \text{ degré})=0.5$, donc 55.66 km/degré

La formule du cosinus compense la convergence vers les pôles

source

02/10/2025

Git

je devrais créer les issues + milestones, fait avant la fin de la semaine.

04/10/2025

Relief

Commencer avec la factory + interface

05/10/2025

Git

issues + milestones done.

08/10/2025

Relief

Impossible de trouver un DSM (digital surface model) pour l'Europe, la France ou l'Allemagne, car elles ont soit des maillages trop grosses (30m ou 50m par exemple), soit je ne peut pas avoir accès pour cause de coûts ou d'accès privés aux institutions publiques et autres.

Je ne peux pas utiliser un DEM (digital elevation model) parce qu'il me faut les objets comme les forêts et ponts et bâtiments pour raison de sécurité et justesse.

09/10/2025

Relief

Autre carte: swissTLM3D mais pas d'API, seulement un bulk download disponible. Pensait utiliser TLM avec ALTI pour avoir des informations sur le type de terrain

+ l'élévation pour par exemple avoir un plus grand delta au dessus des zones de forêts.

10 - 23 /10/2025

Relief

J'ai conçu la factory + interface provider d'élévation.

J'ai programmé le simulateur de provider pour créer une matrice en se basant sur une résolution donnée et une taille prédéterminée. Permet de "fetch" la hauteur d'un point en utilisant l'interpolation bilinéaire pour se rapprocher d'un point dans la matrice (index ligne / colonne).

26/10/2025

Swisstopo api marche maintenant

28/10/2025

Programme QGIS permet de parser les GeoTIFFs donc avec ça j'ai pu confirmé que les tiles téléchargés par le code étaient bien correct.

Problème maintenant: comment parser le format GeoTIF pour bien faire marcher pour le programme (Algo1D)?

En première temps, il faudrait convertir en matrice et y rajouter les points de hauteurs.

Donc on créer un ElevationData avec les points d'hauteurs de l'image TIFF.

29/10/2025

Implémentation d'un loader de tile GeoTIFF, très simple. J'ai aussi changé les noms des tiles téléchargés à leur bounding box

Le canal rouge du pixel du geotiff contient l'hauteur.

Un bug, matrice contient des valeurs nulles. Faudrait voir pourquoi.

Des 4 millions de pixels, 2% ont des valeurs nulles, donc il faudrait faire de l'interpolation pour les corriger.

Remarque: les tiles sont téléchargé mais il faudrait checker si les tiles avec les correctes bounding box sont téléchargé au stade pre-flight, sinon on les enlève (idée de cache comme ça l'utilisateur qui vole au meme endroit à déjà les tiles).

30/10/2025

Tester est facile. Avec les unit tests (qu'on peut grouper pour former des functional tests), on peut just lancer `flutter test` pour tout tester.

Bug: les hauteurs ne matchent pas les points GPS (tester avec le Soliat à 1464 mètres). Recherche de problème:

1. Voir bounding box
2. Voir `getElevationGPS`
3. Voir `getElevation`
4. Voir téléchargement de tile
5. Voir nom bounding box des fichiers tiles

31/10/2025 - 02 /11/2025

Recherche du bug...

Changement vers l'éditeur "Zed" pour programmer le projet, il aime bien linter le code, donc les fichiers sont un peu plus lisible maintenant

03/11/2025

Bug: Dans GeoTiff loader, il faut changer les coordonnées des pixels:

```
elevations[i * elevationsImage.width + j] = elevationsImage
    .getPixel(i, j)
```

vers

```
elevations[i * elevationsImage.width + j] = elevationsImage
    .getPixel(j, i)
```

Les coordonnées du Soliat nous donne 1462.6 mètres, donc très proche du 1464 mètres inscrit sur les cartes.

Autre bug: dans l'unit test, il prend le premier fichier puis en créer le bounding box, il faudrait donc qu'il cherche le fichier correspondant au gps actuel. Ou peut-être tout avoir en mémoire? ou seulement avoir en mémoire les tiles entre le parapentiste et le waypoint?

Pour l'instant, on va plutôt mettre tout dans la mémoire

Petit calcul: 100 tiles, chacun 16 MB, alors 1.6 GB en mémoire vive...

Pour l'algo1D, avoir que les tiles entre l'utilisateur et le waypoint est logique car c'est une fonction linéaire, mais ceci ne marchera pas pour l'algo2D

04/11/2025

J'ai consacré quelques heures à implémenter la grosse matrice mais je pense il ne faut pas faire ça comme ça. Je pense que c'est l'algorithme qui devrait prendre en charge quel tile utilisé lors de son trajet.

05/11/2025

Algo1D

Longitude, latitude et altitude sont respectivement x,y,z. Dans la littérature, ils utilisent d'autres symboles mais ceci nous aide pour la compréhension. De plus, il faut convertir en radian le degré de latitude et longitude.

Il faut tenir compte de la courbure de la Terre, donc nous allons utiliser la formule de haversine pour calculer la distance correctement:

$$\Delta x = x_{wp} - x_{user} \quad \Delta y = y_{wp} - y_{user}$$

$$a = \sin^2(\Delta y/2) + \cos y_1 \cdot \cos y_2 \cdot \sin^2(\Delta x/2) \quad c = 2 \cdot \arctan 2(\sqrt{a}, \sqrt{(1-a)}) \quad d(x, y) = R \cdot c$$

Avec $R = 6,371 \cdot 10^6$ mètres, comme défini pour l'ellipsoïde du système de coordonnées WGS84

Codage de l'algorithme sans obstacles

06/11/2025

Algo2D

Liste d'algorithme qui pourrait être utilisé pour le contournement:

- A*
- Theta (*variante de A* basé sur "Line of Sight")
- Génétique
- Heuristic
- Dijkstra

En considération, il faudrait que l'utilisateur tourne le moins possible pour préserver au plus le ratio de la finesse (car tourner fait perdre de l'énergie).

Donc par exemple:

1. Algo A* qui donne une trajectoire initiale
 2. Algo de funneling pour rendre la trajectoire plus potable pour le parapentiste
- :

- (a) <https://medium.com/@reza.teshnizi/the-funnel-algorithm-explained-visually-41e374172d2d>
- (b) <https://digestingduck.blogspot.com/2010/03/simple-stupid-funnel-algorithm.html>
- 3. On prend en compte la dégradation de la finesse lors des actions de manoeuvres et un safety margin assez grand (ex: >25m) (une méthode de loss?)

Il y a aussi l'algo de Dubin's Path

11/11/2025

Algo1D

Commencer à programmer algo avec obstacle

Je vais diviser la ligne entre le parapentiste et le waypoint en n points équidistants de 0.5 mètres (le maillage).

A chaque point, on va tester l'altitude du point algorithmique avec l'hauteur du tile correspondant.

On retourne la différence la plus basse.

12/11/2025

Algo1D

Continuation

Rapport

Commencé à le rédiger

14-17/11/2025

Algo1D

continuation

17/11/2025

Discussion avec Bilat:

1. Pas besoin de marge, il faut être très précis dans notre calcul et c'est à l'utilisateur d'estimer la marge nécessaire
2. Faire des plots pour le temps de chargement des tiles

3. Faire un pie graph pour voir où le bottleneck de performance pourrait être (chargement tile, algo, etc.)
4. Tester avec des matrices simulées

18/11/2025

Algo1D

equation form:

$$\Delta h_{climb} = \max \left(0, -\min \left[h_{terrain}(i) + \frac{d_i}{f} - h_{user} \right] \right)$$

Pour chaque itération, nous allons calculer le delta d'elevation entre le point i et le terrain relatif à l'altitude de l'utilisateur

Gros problème de performance car il charge les fichiers chaque fois qu'un nouveau point va comparer son delta avec un tile, donc meilleur solution: charger en mémoire les tiles depuis l'utilisateur jusqu'au waypoint. Autre problème de performance? Reste à voir, peut-être il faut paralléliser.

Sinon l'algo marche.

19-20-20/11/2025

Algo1D

Il faudrait coder un tile controller qui va dynamiquement chargé et déchargé les tiles. Ils se trouvent dans une map donc les tiles pertinentes sont dorénavant toujours en mémoire le long du trajet, comme ça les chargement seront pas autant intensif sur le système (car dans mémoire vive).

Nous allons utilisé l'algorithme de tracé de segment de Bresenham pour déterminé les tiles que nous devront chargé. Les tiles vont être vu comme des "pixels".

Bresenham est rapide car il utilise des nombres entier pour iterativement trouvé les pixels qui sont segmenté par la ligne, en tenant compte de l'accumulation d'erreur chaque fois que x est incrémenté. Si l'erreur est plus grande que la distance depuis le sommet d'un pixel, alors il faut incrémenter y et reformuler l'erreur pour le nouveau pixel.

1. Delta absoluts

- $dx = |x_{End} - x_{Start}|$
- $dy = |y_{End} - y_{Start}|$

2. Direction du segment (s_x, s_y)
 - Si $x_{\text{Start}} < x_{\text{End}}$, alors $s_x = 1$, sinon $s_x = -1$
 - Si $y_{\text{Start}} < y_{\text{End}}$, alors $s_y = 1$, sinon $s_y = -1$
3. Initialisation de l'erreur accumulative
 - $\varepsilon = dx - dy$
4. Boucle de l'équation
 - (a)
 - Si $2 \cdot \varepsilon > -dy$
 - $\varepsilon = \varepsilon - dy$
 - $x = x + s_x$
 - Si $2 \cdot \varepsilon < dx$
 - $\varepsilon = \varepsilon + dx$
 - $y = y + s_y$

Ceci veut aussi dire que nous devons changer les noms des fichiers car nous allons convertir en "grid" puis reconvertir en nom de fichier, ce qui va être problématique avec la solution actuelle qui est d'utiliser le bounding box avec lat lon en nombre flottant. Ça nous va bien, car nous pourrions construire un "lookup table" avec un map qui va faciliter la performance. Nous pouvons donc voir en $O(1)$ si un tile est dans la mémoire ou pas.

On peut encore se faciliter la vie, car l'api nous donne déjà les tiles en grid, donc les noms des fichiers vont avoir ça et leur bounds. Il reste donc à avoir un outil pour convertir les latitudes et longitudes en LV95 et vice versa.

Format nom fichier: x-y-minlat-maxlat-minlon-maxlon.tif

En coordonnées LV95, x représente les kilomètres depuis l'Est, alors qu'y représente les kilomètres depuis le Nord. Vu que les tiles sont de $1km^2$ et que le grid sont des entiers, on a moins d'erreurs qu'avec notre situation d'avant.

Donc c'est comme ça que nous allons programmer le tile controller

22/11/2025

J'ai fait:

- Le contrôleur de tile en mémoire
- Le contrôleur principale du programme en real time
- L'algo1D marche avec les tiles en mémoire

- Rajouter un manifeste dans `android/app/src/main/AndroidManifest.xml` pour expliciter les permissions nécessaire pour le package `geolocator`
- Commencer à travailler avec un émulateur qui permet de spoofer la location GPS donc très utile (aussi permet de travailler avec les dev tools pour profiler la performance)
- petit ui pour l'application sur mobile
- fixer le downloader car l'API limite jusqu'à 100 tiles par lien mais il y a un lien "next" pour récupérer les prochains tiles donc ça c'est fixé

Prochainement:

- Il faut tester plus
- Il faut créer des units tests pour tous les nouvelles choses programmées
- etc.

Bug: lookup grid lv95

25/11/2025

bug corrigé. Tile controller va maintenant prendre les tiles autours de chaque tile dans le path pour éviter les erreurs. Encore performant.

Autre bug: altitude gps sur natel n'est pas précis

29/11/2025

looked into generating documentation in Dart

03/12/2025

Fineness peut être sélectionner maintenant.

Optimization: si un tile est déjà présent dans le dossier des tiles avant le download_screen et que l'on va le réutiliser, alors le downloader ne va pas le retélécharger

Algo2D

idée:

A* depuis le waypoint jusqu'à l'utilisateur avec une finesse inversé.

Si l'utilisateur est en dessous du point final, alors l'algo lui dit de combien il faut monter.

Si l'utilisateur est en dessus, alors il va en plus faire un algorithme de funnel puis lui dire quel orientation suivre chaque x seconde.

04/12/2025

https://www.researchgate.net/publication/367477023_Aircraft_Emergency_Trajectory_Design_A_Fast_Marching_Method_on_a_Tetrahedral_Mesh

Très similaire au projet

Utilise cet algorithme: <https://stanfordasl.github.io/wp-content/papercite-data/pdf/Janson.Pavone.ISRR13.pdf>

05-11/12/2025

Compréhension de l'algorithme.

Décider de ne pas l'implémenter, pas assez de temps.

12-17/12/2025

Analyse du temps d'algorithme, du plottage du chemin de Bresenham et temps de téléchargement

20/12/2025 - 05/12/2025

Autres analyse

Fin du projet

Implémentation rapide de la voix et de l'altitude au waypoint (en parlant avec Bilat), calibration pour offset du GPS et beaucoup de bug fixing and de commentaires, etc.

E User guide

With the results seen in the main document, we can formulate a user guide:

App install

1. Put the app-release.apk in a folder on target phone
2. Allow permission to install unknown apps
3. Install
4. Open the app

Before flight

1. Pick a waypoint
2. Choose fineness ratio
3. Choose wander radius
4. Wait for tiles to download
5. Get within wander range (the take off site)
6. Set altitude on ground at take off
7. Press launch, then start, and wait for first run of the algorithm to conclude
8. Take off.

Flight

1. Every 10 seconds, listen to the climb needed to reach the waypoint, and what the altitude of the waypoint will be after climbing.
2. When finished flying, simply close the application.

F Code documentation

The full API documentation generated by Dart's docs method is available in the folder : 030-Developpement/doc/. Click on the index.html file for an overview.

A UML diagram can be found in the PDF: 010-RapportFinal-Annexes/03_UML.pdf